IIT-H, Deep Learning 2026, Hackathon

Final Submission Report

Team Name: **ASV**

M M Ajeya Roll No.: AI25MTECH14005
Soham Kamat Roll No.: AI25MTECH14011
Vineeth Rambhiya Roll No.: AI25MTECH11005

May 2, 2026

Abstract

This report describes our approach for the Kaggle Deep Learning 2026 Hackathon. The task involved classifying synthetic CLEVR-like images into two classes. Initial visual analysis suggested strong correlations between class labels and attributes such as color, material, and object size. However, experiments showed that these attributes were spurious and did not generalize well to the Kaggle test set. We therefore shifted to a structure-based approach based on counting the number of spheres, cubes, and cylinders in each image. The final solution used a YOLOv8 model trained from scratch on CLEVR data to directly detect and classify object shapes. The detected shape counts were then mapped to the final class label using the tuple distribution observed in the training set. This approach achieved a public leaderboard score of **93.3%**.

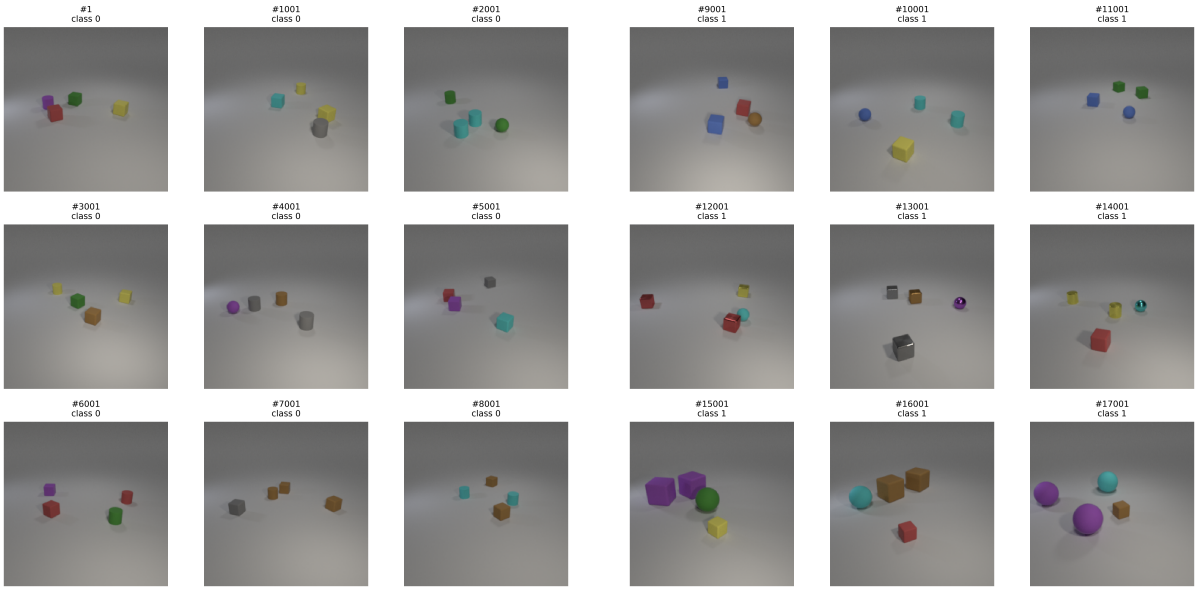
1 Introduction

The hackathon task was to classify synthetic images into two classes. Each image contained exactly four foreground objects placed on a simple background. The objects belonged to one of three possible shape categories: sphere, cube, or cylinder.

At first, the training data appeared to contain simple visual patterns separating the two classes. In particular, class 1 images seemed to contain blue objects, metallic objects, or large objects. However, further experimentation showed that these visual features were not sufficient for generalization. This motivated a shift from appearance-based classification to a more structured shape-counting approach.

2 Initial Dataset Observations

We first manually inspected examples from class 0 and class 1. Representative samples are shown in Fig. 1. A larger collection of inspected training examples used during this analysis is provided in Appendix A, Subsection A.1.



(a) Class 0 examples

(b) Class 1 examples

Figure 1: Representative examples from class 0 and class 1. Larger image grids used for manual inspection are provided in Appendix A.1.

From visual inspection of these examples and the broader image grids in Appendix A.1, we observed the following structure in the class 1 samples:

- The first 3000 class 1 images frequently contained a blue object.
- The next 3000 class 1 images frequently contained a metallic object.
- The last 3000 class 1 images frequently contained a large object.

This led to our first hypothesis that the class label was determined by the presence of blue, metallic, or large objects.

3 Experiment 1: Attribute-Based ResNet Classifiers

To test the visual-attribute hypothesis, we trained three separate ResNet-based binary classifiers:

1. A blue-object detector.
2. A metallic-object detector.
3. A large-object detector.

For each classifier, we created a balanced dataset using 3000 class 0 images and the corresponding 3000-image class 1 subset. The data was split into 50% training, 25% validation, and 25% testing. Horizontal flip augmentation was applied only to the training split, effectively doubling the number of training samples while leaving the validation and test sets unchanged. This setup was chosen to encourage generalization despite the limited subset size.

Table 1: Validation accuracy of the attribute-based ResNet classifiers.

Model	Validation Accuracy
Blue detector	1.0
Metallic detector	1.0
Large-object detector	1.0

Although these models achieved high validation accuracy, the combined approach achieved only **75%** accuracy on the Kaggle public leaderboard. This indicated that color, material, and size were likely spurious correlations rather than the true classification rule.

4 Experiment 2: Removing Spurious Visual Features

To reduce the effect of color, material, and size, we tried preprocessing-based approaches.

4.1 Grayscale Conversion

The first approach was to convert images to grayscale. This removed explicit color information, especially the blue-object cue. However, metallic objects were still visually distinguishable because their brightness and reflectance patterns remained visible.

4.2 Binary Foreground Masks

We then tried converting images into binary foreground masks so that color and material information would be suppressed. However, this introduced another problem: gray objects often blended into the gray background, causing unreliable foreground extraction.

This approach achieved only **73%** on the Kaggle leaderboard. Therefore, simple image preprocessing was not sufficient to remove spurious visual cues while preserving useful structural information.

5 Hypothesis Shift: Shape-Count Based Classification

After the failure of the appearance-based approaches, we reconsidered the possible hidden rule. Since every image contained exactly four objects, and each object belonged to one of three shapes, we hypothesized that the class label may depend on the tuple:

$$(\#\text{spheres}, \#\text{cubes}, \#\text{cylinders}).$$

There are 15 possible tuples for distributing four objects among three shape categories. This suggested that the problem could be solved by identifying the number of spheres, cubes, and cylinders in each image.

6 Experiment 3: Direct 15-Way ResNet Classifier

Since the competition rules allowed the use of external data, we leveraged the CLEVR v1.0 dataset to train a shape-count classifier. We filtered CLEVR scenes to images containing exactly four objects, yielding **8,664 training** and **1,925 validation** images. The label for each image was the index of its (cubes, spheres, cylinders) triple in the 15-class enumeration of non-negative integer triples summing to four.

We trained a **ResNet-34 from scratch** (no pre-trained weights, final FC replaced with a 15-way head; $\sim$11.2M parameters) for 15 epochs.

Table 2: Results of the direct 15-way ResNet-34 classifier (CLEVR, 4-object subset).

Metric	**Value**
Training accuracy (epoch 15)	98.72%
Validation accuracy (best)	97.35%
Kaggle public score	**81%**

Mapping tuples to binary labels. The hackathon task is binary, but our classifier predicts one of 15 tuples. To assign a binary label, we used the labelled hackathon *training* set: for every shape-count tuple that occurred there, we recorded which class (0 or 1) it appeared with most frequently, and adopted that majority class as the label for that tuple. At inference time, we ran the ResNet-34 on each test image, took the argmax tuple, and looked up its assigned class. With this scheme we obtained a Kaggle public-leaderboard score of **81%**.

The result supported our hypothesis that shape counts carry the relevant signal, but the direct 15-way formulation has a structural limitation: all errors are weighted equally. For example, confusing $(1, 2, 1)$ with $(2, 1, 1)$ is a much smaller miscount than confusing $(1, 2, 1)$ with $(4, 0, 0)$, yet the cross-entropy loss penalizes both identically. This motivated our move to an object-detection-based approach, which scores each shape independently and degrades gracefully when the model is off by a single object.

7 Final Approach: YOLO-Based Direct Shape Counting

Initially, we considered a two-stage pipeline:

1. Use YOLO to detect object bounding boxes.
2. Crop the detected objects and pass them to a separate classifier to classify each object as a cube, sphere, or cylinder.

However, after training YOLO directly on CLEVR with shape-specific labels, we observed that YOLO itself was able to localize and classify the objects with very high accuracy. On the training distribution, the detected shape tuples produced a perfect class split. Therefore, the second-stage crop classifier was not required.

The final pipeline was:

1. Run YOLO on each image.
2. Select the four highest-confidence detections.
3. Count the number of detected spheres, cubes, and cylinders.
4. Map the resulting tuple to class 0 or class 1.

7.1 Preparing YOLO Labels from CLEVR

CLEVR provides object centroids (`pixel_coords`), depth, size, and shape annotations, but does not include YOLO-compatible bounding boxes. We therefore reconstructed approximate 2D bounding boxes analytically using a pinhole-camera projection model.

For an object of 3D radius r at depth d, the projected half-size in pixels was estimated as:

$$\text{half_pixels} = \frac{fr}{d},$$

where f denotes the focal length in pixels. We used CLEVR's standard image resolution of 480×320, object radii of 0.7 for large and 0.35 for small objects, and an approximate focal length of 525 pixels.

Since cubes and cylinders do not project identically to spheres, shape-specific width and height multipliers were introduced to better approximate their silhouettes, as shown in Table 3. These values were chosen heuristically based on CLEVR geometry and typical viewing angles.

Table 3: Shape-specific bounding-box multipliers used for YOLO label generation.

Shape	**Width Multiplier**	**Height Multiplier**
Sphere	1.0	1.0
Cube	1.15	1.15
Cylinder	1.0	1.6

The resulting boxes were centered at the provided object coordinates, clipped to image bounds, and normalized to standard YOLO format:

$$\text{class_id},\ x_{\text{center}},\ y_{\text{center}},\ w,\ h.$$

Although these boxes are approximate rather than exact rendered masks, they proved sufficiently accurate for training the downstream YOLO detector.

7.2 YOLO Training Setup

We trained YOLOv8n from scratch on the generated CLEVR labels. No pretrained weights were used. The model was instantiated from the YOLOv8n YAML configuration and trained with random initialization.

Table 4: YOLOv8 training configuration.

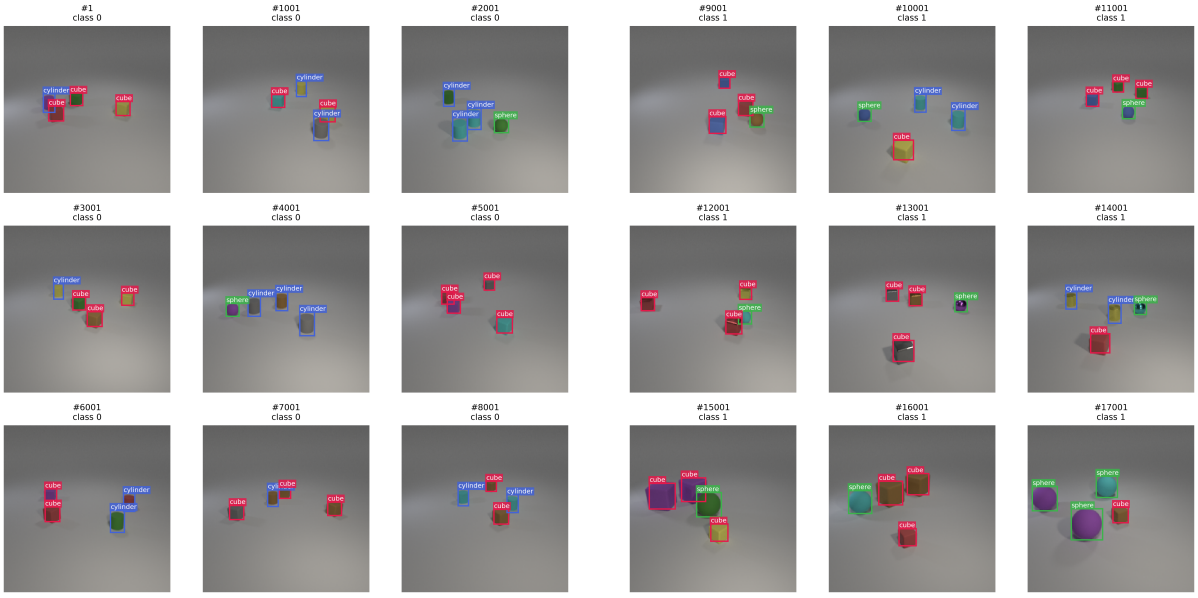
Parameter	**Value**
Model	YOLOv8n
Initialization	Random initialization
Pretrained weights	No
Epochs	80
Image size	480
Batch size	128
Optimizer	SGD
Initial learning rate	0.01
Momentum	0.937
Weight decay	5×10^{-4}
Learning-rate schedule	Cosine LR
AMP	Enabled
HSV (h, s, v) jitter	0.0, 0.0, 0.2
Translate / scale	0.05 / 0.2
Degrees / shear / perspective	0.0 / 0.0 / 0.0
Horizontal flip	0.5
Vertical flip	0.0
Mosaic augmentation	1.0
Mixup	0.0

Mosaic augmentation was kept enabled to encourage robustness to object scale and location. Heavy photographic augmentations were disabled because CLEVR images are synthetic and visually clean; only a mild value-channel perturbation, small translate/scale jitter, and horizontal flipping were retained.

Table 5: YOLO validation results (CLEVR val split, 15,000 images).

Metric	**Value**
Box loss	0.164
Classification loss	0.097
Precision	0.9999
Recall	0.9998
mAP@50	0.995
mAP@50–95	0.9936

The quantitative validation metrics in Table 5 indicate that the detector achieves near-perfect localization and classification performance on the CLEVR validation split. Qualitative detection examples on representative hackathon images are shown in Fig. 2, where the trained YOLO model accurately localizes each object and predicts the correct shape label across both class 0 and class 1 samples.



(a) Class 0 examples (b) Class 1 examples

Figure 2: Example YOLO detections showing predicted bounding boxes and shape labels.

7.3 Tuple-to-Class Mapping

After obtaining shape-count tuples, we computed the tuple distribution in the hackathon training set. We observed that most tuples appeared exclusively in one class. The final class was therefore assigned using the tuple-to-class mapping shown in Table 6.

Table 6: YOLO-predicted shape-count tuple distribution on the training set. Each observed tuple appears exclusively in either class 0 or class 1, suggesting highly accurate YOLO predictions and a strongly separable tuple-to-class mapping.

Tuple: (spheres, cubes, cylinders)	**Class 0 Count**	**Class 1 Count**
(0,0,4)	151	0
(0,1,3)	1385	0
(0,2,2)	2998	0
(0,3,1)	2476	0
(0,4,0)	743	0
(1,0,3)	586	0
(1,1,2)	0	3553
(1,2,1)	0	0
(1,3,0)	0	4085
(2,0,2)	515	0
(2,1,1)	0	0
(2,2,0)	0	0
(3,0,1)	135	0
(3,1,0)	0	1362
(4,0,0)	11	0

Three tuples were not present in the training set:

$$(1, 2, 1), \quad (2, 1, 1), \quad (2, 2, 0).$$

To better visualize the geometric structure of the tuple distribution, Fig. 3 plots all valid shape-count combinations in the 3D tuple space. The observed class 0 and class 1 tuples occupy distinct regions, while the three missing tuples lie spatially closer to the class 1 region than the class 0 region.

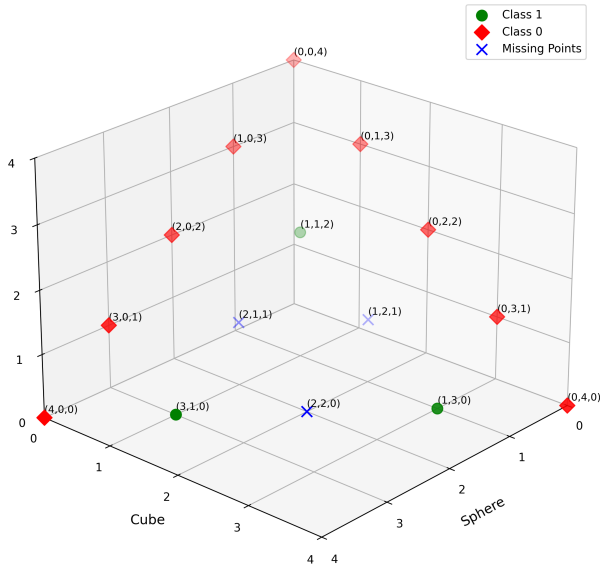


Figure 3: 3D visualization of the YOLO-predicted tuple distribution. Red diamonds denote tuples observed in class 0, green circles denote tuples observed in class 1, and blue crosses denote tuples absent from the training set.The unseen tuples lie within the geometric continuation of the class 1 region, motivating their assignment to class 1.

Based on the observed structure of the tuple distribution and the geometric placement of the missing tuples in Fig. 3, we assigned all three unseen tuples to class 1 in the final submission.

7.4 YOLO-Based Final Pipeline Results

Using the tuple-to-class mapping described above, the complete YOLO-based direct shape-counting pipeline produced a perfect class split on the hackathon training set and achieved a public Kaggle leaderboard score of **93.3%**.

This strong performance indicates that the learned object detector generalized effectively from CLEVR to the hackathon dataset and that the shape-count tuple representation captures the dominant decision structure underlying the hidden classification rule.

The leaderboard performance of the final submission is provided in Appendix A.A.2.

8 Summary of Results

The final YOLO-based direct shape-counting pipeline produced a perfect class split on the training data and achieved a public Kaggle score of **93.3%**. The main experimental results are summarized in Table 7.

Table 7: Comparison of attempted approaches.

Approach	**Observation**	**Kaggle Public Score**
Attribute-based ResNet classifiers	Learned blue, metallic, and large-object cues, but these were spurious	75%
Grayscale / binary-mask preprocessing	Reduced some visual cues but caused foreground-background ambiguity	73%
Direct 15-way ResNet-34 classifier	Validated shape-count hypothesis but still had tuple-level classification errors	81%
YOLO-based direct shape counting	Detected object shapes directly and mapped count tuples to labels	**93.3%**

9 Conclusion

Through a series of experiments, we observed that the most visually apparent features in the dataset (namely blue color, metallic material, and large object size) were not the true determinants of the class labels, despite yielding high validation accuracy on selected training subsets. Their poor leaderboard performance indicated that these were spurious correlations present in the training distribution.

This motivated a shift from appearance-based classification toward a structural understanding of the scene. By hypothesizing that the label depended on the count of spheres, cubes, and cylinders in each image, we reformulated the task into a shape-counting problem. A direct 15-way classifier on shape-count tuples provided initial evidence supporting this hypothesis, but the final YOLO-based approach proved more robust and interpretable.

Our final pipeline trained YOLOv8n from scratch on CLEVR data, directly detected and classified object shapes, counted the number of spheres, cubes, and cylinders, and mapped the resulting tuple to the final class label. Since YOLO itself provided sufficiently accurate shape classification, an additional second-stage classifier was unnecessary, keeping the pipeline simple and efficient.

Overall, this method achieved a public leaderboard score of **93.3%**.

A Appendix

A.1 Extended Training Image Inspection

To better understand the underlying class structure, we manually inspected a larger collection of training images beyond the representative examples shown in the main text. These broader grids were used to identify recurring visual patterns and formulate the initial hypothesis regarding blue, metallic, and large-object correlations.

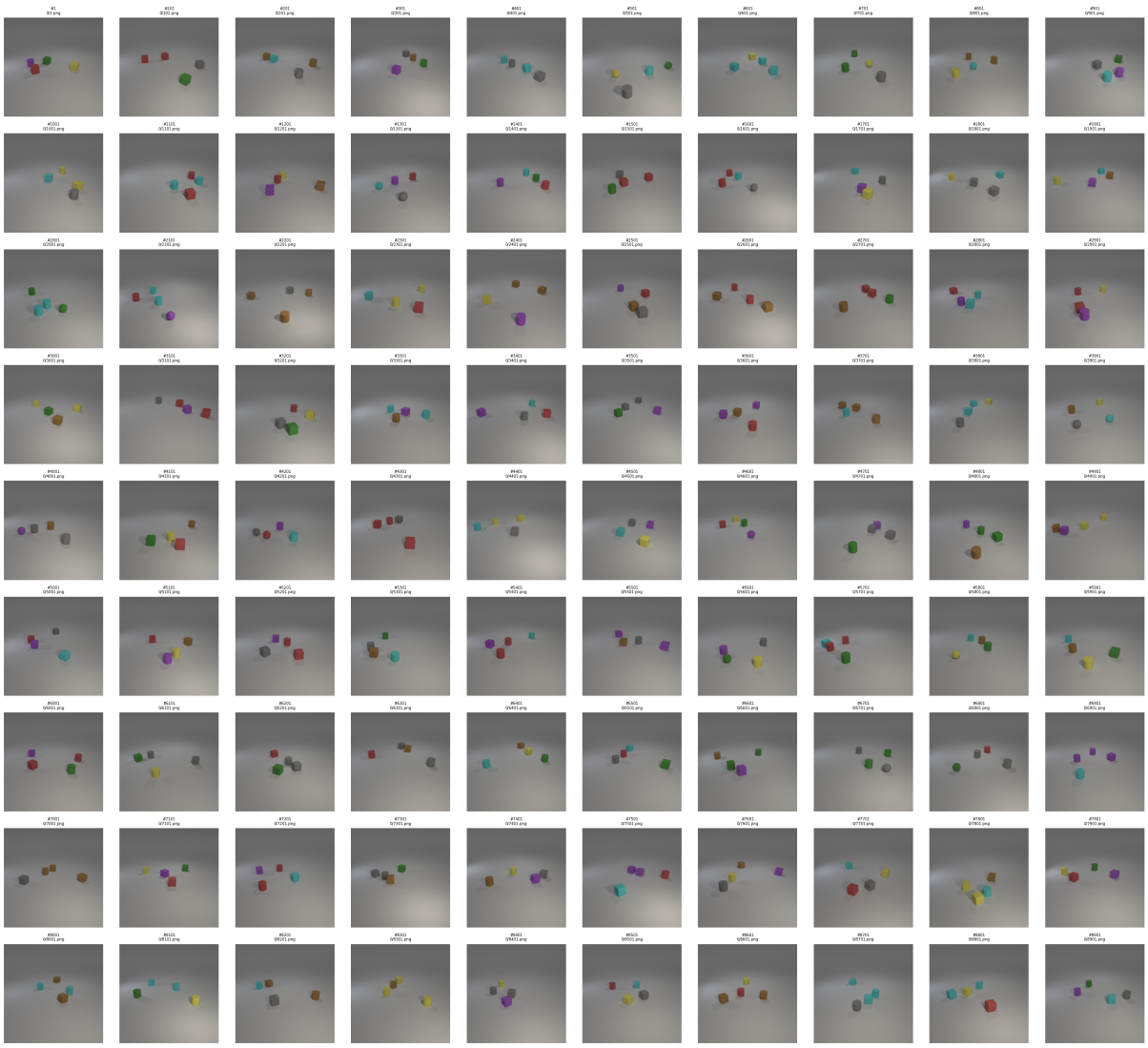


Figure 4: Extended grid of inspected class 0 training samples.



Figure 5: Extended grid of inspected class 1 training samples.

A.2 Kaggle Public Leaderboard Performance

Figure 6 shows the public leaderboard ranking achieved by our final submission at the time of submission. Our team (**ASV**) achieved a public leaderboard score of **93.34%**, placing **2nd** on the leaderboard.

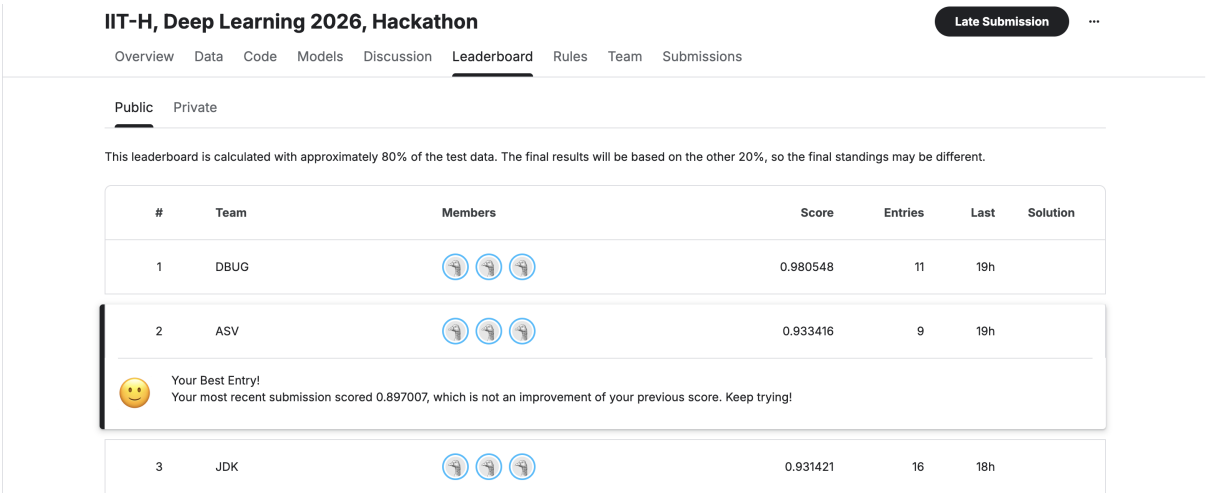


Figure 6: Public Kaggle leaderboard showing the performance of the proposed method. Team ASV achieved 2nd place with a score of 93.34%.

A.3 Team Contributions

The contributions of each team member toward the final submission are summarized below.

- **M M Ajeya (AI25MTECH14005):** Led the overall solution design and implementation, including dataset analysis, experimental design, model development, result analysis, and report writing.
- **Soham Kamat (AI25MTECH14011):** Contributed to exploratory data analysis, hypothesis brainstorming, experimental validation, solution discussions, and report writing.
- **Vineeth Rambhiya (AI25MTECH11005):** Contributed to exploratory data analysis, hypothesis brainstorming, experimental validation, solution discussions, and report writing.